High-Performance 24-Hour Timeline Control in WinCC V8

> **Subject:** System Architecture, Data Pipeline, and Codebase Reference
> **Target Environment:** Siemens WinCC V8, Custom Web Controls (HTML5/CSS3/JS), OLEDB Provider
> **Author:** Kamal Ranjbar | Industrial Automation Engineering

1. Executive Summary & Architecture Overview

Traditional SCADA systems like WinCC V8 often impose visual and functional limitations when displaying multi-state historical timeline graphics (e.g., 24-hour operation logs for industrial filter presses). To overcome these constraints, a robust **3-tier hybrid ETL (Extract, Transform, Load) architecture** was engineered.

This solution bridges legacy runtime databases with modern web-based UI controls, ensuring real-time performance, responsive scaling via `ResizeObserver`, and reliable active duration monitoring.

2. The 3-Tier Data Pipeline

The system decouples data extraction from frontend rendering across three specialized layers:

- **Tier 1 - Storage & Archiving:** Raw operational status tags (e.g., `_WTC_Mode`) are captured by the PLC and logged into WinCC Tag Logging archives on every state change.
- **Tier 2 - Backend ETL Bridge (VBScript):** Triggered by screen `OnOpen` events, a VBScript queries the OLEDB database, serializes historical state logs into compact token strings, and buffers them into internal WinCC text tags (`_ModeData1-4`).
- **Tier 3 - Frontend UI (Custom Web Control):** An embedded HTML5/JavaScript engine reads the text tags, parses tokens, maps temporal positions, and renders a dark-themed interactive timeline.

3. Backend ETL: VBScript & OLEDB Extraction

Upon navigating to the HMI view, the screen's `OnOpen` event executes an optimized VBScript routine to extract archive records. It bypasses string length constraints through smart tokenization.

3.1. OLEDB Connection & Query Routine

```
Dim conn, rs, sql, catalogName
catalogName = HMIRuntime.Tags("@DatasourceNameRT").Read
Set conn = CreateObject("ADODB.Connection")
conn.ConnectionString = "Provider=WinCCOLEDBProvider.1;Catalog=" & catalogName & ";Data Source=.\WinC
conn.Open

sql = "TAG:R,'ArchiveName','FilterPress1_WTC_Mode'..."
Set rs = conn.Execute(sql)
```

3.2. Data Serialization & Tokenization

Historical state transitions are compressed into 7-character fixed tokens (1 digit for mode ID, followed by 6 digits for HHMMSS timestamp). Since WinCC text tags limit character capacity, strings are split across four buffer tags:

```
' Token structure: [Mode ID (1)][Hour (2)][Minute (2)][Second (2)]
tok = CStr(modeVal) & Right("0" & Hour(ts), 2) & Right("0" & Minute(ts), 2) & Right("0" & Second(ts),
bufferChunk = bufferChunk & tok

' Writing to internal tags
HMIRuntime.Tags("_ModeData1").Write chunk1
HMIRuntime.Tags("_ModeData2").Write chunk2
```

4. Frontend Engine: Custom Web Control (HTML5 & JavaScript)

The custom web control interfaces with WinCC runtime tags via the official Siemens `webcc.min.js` library, parsing incoming payload data asynchronously.

4.1. WebCC Property Binding & Parsing

```
WebCC.start(function(result) {
    if (result) {
        var m1 = WebCC.Properties.modeData1 || "";
        var m2 = WebCC.Properties.modeData2 || "";
        var m3 = WebCC.Properties.modeData3 || "";
        var m4 = WebCC.Properties.modeData4 || "";

        var rawPayload = m1 + m2 + m3 + m4;
        parseTimelineTokens(rawPayload);
    }
}, ['status', 'modeData1', 'modeData2', 'modeData3', 'modeData4']);
```

4.2. Token Deserialization Logic

The client-side engine reconstructs operational intervals from the concatenated token string:

```
function parseTimelineTokens(payload) {
    var segments = [];
    for (var i = 0; i < payload.length; i += 7) {
        var token = payload.substr(i, 7);
        if (token.length < 7) break;
        segments.push({
            mode: parseInt(token.charAt(0)),
            time: {
                h: parseInt(token.substr(1, 2)),
                m: parseInt(token.substr(3, 2)),
                s: parseInt(token.substr(5, 2))
            }
        });
    }
    renderTimelineDOM(segments);
}
```

5. Responsive Scaling via ResizeObserver

To eliminate initial render clipping or sizing bugs common in WinCC container frames, a `ResizeObserver` monitors container boundaries dynamically:

```
const observer = new ResizeObserver(entries => {
    for (let entry of entries) {
        recalculateAndRedrawTimeline();
    }
});
observer.observe(document.getElementById('app-container'));
```

6. Troubleshooting & Operational Reference

Symptom	Root Cause	Resolution
Timer resets on page navigation	WinCC picture caching preventing `OnOpen` re-execution	Verify graphics designer picture cache settings and script triggers.
Timestamp skew / offset	Windows OS timezone or Daylight Saving Time mismatch with OLEDB	Synchronize local Windows time configuration and regional settings.

7. Conclusion

This hybrid architecture successfully circumvents SCADA UI bottlenecks, delivering high performance and maintainability for industrial environments.